# Functions

# Functions

- what is new?
- Multiple return values
- Named result parameters
- Defer

# Function: declaration

```
func functionname(parametername type) returntype {
 //function body
}
```

The parameters and return type are optional in a function. Hence the following syntax is also a valid function declaration.

```
func square(f float64) float64 {
      return f*f }
```

```
func ciao()  {
      fmt.Println("Ciao")
}
```

```
func add(x, y int) int {
      return x + y
}
```

# Pass by Value

**All arguments in Go are passed by value**.

- This means the function receives a **copy** of the variable, not the original.

```
func increment(x int) {
    x++
}
func main() {
    a := 10
    increment(a)
    fmt.Println(a) // Output: 10
(unchanged)
}
```

```
func increment(x *int) {
    *x++
}


func main() {
    a := 10
    increment(&a)
    fmt.Println(a) // Output: 11
}
```

# Pass by Value reference type

These are **reference types** in Go. When you pass them, the underlying data can be modified because the copy still points to the same data.

```
func modifySlice(s []int) {
    s[0] = 99
}


func main() {
    nums := []int{1, 2, 3}
    modifySlice(nums)
    fmt.Println(nums) // Output: [99 2 3]
}
```

| Type | Reference Type? | Why? |
| --- | --- | --- |
| Slice | ✅ Yes | Holds pointer to array |
| Map | ✅ Yes | Internally a pointer to hash table |
| Channel | ✅ Yes | Internally a pointer to queue |
| Function | ✅ Yes | Points to code and closure |
| Interface | ✅ Yes | Contains pointers to type and data |
| Array | ❌ No | Full copy passed |
| Struct | ❌ No | Full copy passed |

# No default arguments

Go does not support default function arguments.

- too easy to throw in defaulted args to fix design problems
  - encourages too many args
  - too hard to understand the effect of the fn for different combinations of args
- Extra verbosity may happen but that encourages extra thought about names.

Related: Go has easy-to-use, type-safe support for variadic functions.

# Function return type

```
func MySqrt(f float64) (float64, bool) {
        If f >= 0 { return math.Sqrt(f), true }
        return 0, false
        }
```

It is possible to return multiple values from a function.

# Named return values

It is possible to return named values from a function. If a return value is named, it can be considered as being declared as a variable in the first line of the function.

```
func rectProps(length, width float64)(area, perimeter float64) {
    area = length * width
    perimeter = (length + width) * 2
    return  //no explicit return value
}
```

**area** and **perimeter** are the named return values in the above function. Note that the return statement in the function does not explicitly return any value

# Functions with result variables

The result "parameters" are actual variables you can use if you name them.

```
func MySqrt(f float64) (v float64, ok bool) {
        if f >= 0 { v,ok = math.Sqrt(f), true }
        else { v,ok = 0,false }
        return v,ok
}
```

# The blank identifier

What if you care only about the first value returned by MySqrt? Still need to put the second one somewhere.

- Solution: the blank identifier, _ (underscore).

Predeclared, can always be assigned any value, which is discarded.

```
// Don't care about boolean from MySqrt.
val, _ = MySqrt(foo())
```

# The empty return

Finally, a return with no expressions returns the existing value of the result variables.

```
func MySqrt(f float64) (v float64, ok bool) {
 if f < 0 { return }
 return math.Sqrt(f),true
```

```
func MySqrt(f float64) (v float64, ok bool) {
 if  f  >=  0  {  v,ok  =  math.Sqrt(f),  true  }
 return
}
```

# Defer

The defer statement executes a function (or method) when the enclosing function returns. The arguments are evaluated at the point of the defer; the function call happens upon return.

Useful for closing fds, unlocking mutexes, etc.

```
func data(fileName string) string {
  f := os.Open(fileName)
  defer f.Close()
  contents := io.ReadAll(f)
  return contents
}
```

One function invocation per defer

Each defer that executes queues a function call to execute, in LIFO order

```
func  f (){
 for  i  :=  0;  i  <  5;  i++  {
       defer fmt.Printf("%d ", i)
 }
}
```

prints 4 3 2 1 0.

# Function literals

- As in C, functions can't be declared inside functions but function literals can be assigned to variables.

```
func f() {
 for i := 0; i < 10; i++ {
      g := func(i int) { fmt.Printf("%d",i)}
      g(i)
 }
}
```

```
func f() {
 g := func(i int) { fmt.Printf("%d",i)}
 for i := 0; i < 10; i++ {
       g(i)
  }
}
```

# Function literals are closures

```
func adder() (func(int) int) {
    var x int
    return func(delta int) int {
        x += delta
        return x
    }
}
```

clousure

1
21
321

```
func main() {
    f := adder()
    fmt.Println(f(1))
    fmt.Println(f(20))
    fmt.Println(f(300))
}
```

# What is a variadic function?

A variadic function is a function that accepts a variable number of arguments.

If the last parameter of a function definition is prefixed by ellipsis ..., then the function can accept any number of arguments for that parameter.

```
func hello(a int, b ...int) {

}
```

```
hello(1, 2) //passing one argument "2" to b
hello(5, 6, 7, 8, 9) //passing arguments "6, 7, 8 and 9" to b
hello(1) //passing zero argument to b
```

```
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
```

# Empty or blank interface

- interface{} is known as an "empty interface" or "blank interface.»
- It is a type of interface that doesn't specify any methods. In other words, a value of type interface{} can hold a value of any type, as it doesn't require any specific method implementation.

- The empty interface is often used in Go when working with values of unknown type or when writing functions that can accept arguments of any type.
- However, it's important to note that using the empty interface can make the code less type-safe and may result in losing type information during compilation.

```
package main

import "fmt"

func printType(value interface{}) {
    fmt.Printf("Type: %T, Value: %v\n", value, value)
}

func main() {
    printType(42)
    printType("hello")
    printType(3.14)
}
```

# Overload delle funzioni

Soluzione alternativa

- Go do not support function overload

```
package main

import "fmt"

func exampleFunction(args ...interface{}) {
    for _, arg := range args {
        fmt.Println(arg)
    }
}

func main() {
    exampleFunction(1, "hello", 3.14)
}
```

```go
package main

import "fmt"

func exampleFunction(args ...interface{}) {
    for _, arg := range args {
        fmt.Println(arg)
    }
}

func main() {
    // Chiamata con interi
    exampleFunction(1, 2, 3)

    // Chiamata con stringhe
    exampleFunction("hello", "world")

    // Chiamata con float
    exampleFunction(3.14, 2.718)

    // Chiamata con tipi misti
    exampleFunction(42, "go", 3.14, true)
}
```

# Function init

# Package initialization: global identifier

Two ways to initialize global variables before execution of main.main:

- A global declaration with an initializer
- Inside an init() function, of which there may be any number in each source file.

Package dependency guarantees correct execution order.

- Initialization is always single-threaded

- Multiple initialitation

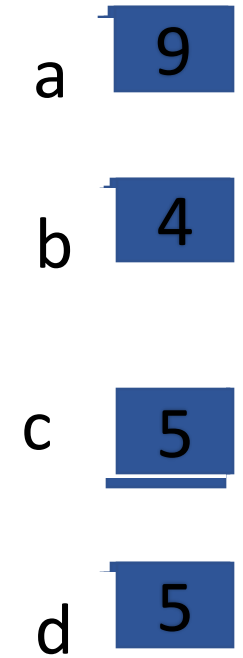
# Init function

- is a **special function** in Go that is **automatically executed** when a package is initialized, **before** main **runs**.
  - It is not called manually and does not return any value.
- It has **no parameters** and **no return values**:
- A single package can have **multiple init functions**, even in different files. They are executed in the order they are defined within a file, and files are initialized in lexical filename order.
- Used to set up **package-level state**, initialize variables, or perform any setup logic before the package is used.
- Package init functions run **after global variables are initialized** but **before main**

```
package main

import "fmt"

var x = initializeX()

func initializeX() int {
    fmt.Println("Initializing x")
    return 42
}

func init() {
    fmt.Println("Init function called")
}

func main() {
    fmt.Println("Main function called")
}
```

# Initialitation example

```
var (
      a = c + b
      b = f()
      c = f()
      d = 3
   )
func f() int {
            d++
            return d
      }
func main() {
      fmt.Println(a)
}
```



# init function

each source file can define its own niladic init function to set up whatever state is required. (Actually each file can have multiple init functions.)

init is called after all the variable declarations in the package have evaluated their initializers, and those are evaluated only after all the imported packages have been initialized.

```
func init()


}
```

a common use of init functions is to verify or repair correctness of the program state before real execution begins.

# init function

- Every package can contain a init function.
- The init function should not have any return type and should not have any parameters.
- The init function cannot be called explicitly in our source code.
- The init function can be used to perform initialisation tasks and can also be used to verify the correctness of the program before the execution starts.
- The order of initialisation of a package is as follows
  - Package level variables are initialised first
  - init function is called next. A package can have multiple init functions (either in a single file or distributed across multiple files) and they are called in the order in which they are presented to the compiler.

# Initialization example

```
package transcendental
import "math"
var Pi float64
func init() {
        Pi = 4*math.Atan(1) // init function computes Pi
}
```

Pi is globally visible

```
package main
import (
	"fmt"
	"transcendental"
	)
var twoPi = 2*transcendental.Pi    // decl computes twoPi
func main() {
	fmt.Printf("2*Pi = %g\n", twoPi)
	}
```

Output: 2*Pi = 6.283185307179586

# Memory management

# Memory management: stack

In Go (Golang), memory management is simplified compared to some other languages, and the division between stack and heap is automatically handled by the runtime. Here's a brief description of how stack and heap are allocated in Go:

Stack:

Stacks are used to manage function calls and store local variables.

Each goroutine in Go has its own stack, which is allocated when the goroutine is created.

Stacks have fixed sizes and are typically smaller than those in some other languages.

This facilitates the quick creation and management of goroutines.

# Escape analysis

```
func escape() *int {
    x := 42
    return &x  // 'x' sfugge (escape), quindi sarà allocata
nell'heap
}


func main() {
    var x int
    go func() {
        fmt.Println(x)  // 'x' sfugge perché la goroutine può sopravvivere alla funzione main
    }()
}
```

# Escape analysis

```
func savePointer(p *int) {
    // Immagina che questo puntatore venga salvato per un uso successivo
}
func main() {
    x := 10
    savePointer(&x)  // 'x' sfugge e viene allocata nell'heap
}
```

# Garbace collector in go

- Before Go 1.5, Go's GC was a **stop-the-world** collector (meaning it paused the entire application during garbage collection).
- With Go 1.5, a **concurrent GC** was introduced, drastically reducing pauses by performing the marking phase in parallel with normal program execution. Since then, subsequent versions of Go have maintained and improved that architecture: **mark-and-sweep**, **tri-color marking**, **write barriers**, and **very short pause times**.
- In Go 1.25, the garbage collector (GC) can operate in two modes: the **"classic" mode** (based on the traditional tri-color mark-and-sweep approach) or—optionally—the new experimental **Green Tea GC**.

# GO GARBACE COLLECTOR: classic mode

Go  uses a garbage collector algorithm called "concurrent mark-and-sweep." This algorithm is designed to minimize the impact on the performance of the running program.

Go's garbage collector operates concurrently, meaning it can work in parallel with the normal execution flow of the program. Additionally, the garbage collector divides its activity into phases, including marking, sweeping, and scanning. The marking phase identifies unused objects, the sweeping phase frees up the memory occupied by these objects, and the scanning phase manages any necessary relocations.

The concurrent approach and distinct phases make Go's garbage collector efficient and suitable for systems that require minimal pause times to ensure quick responses to application requests.

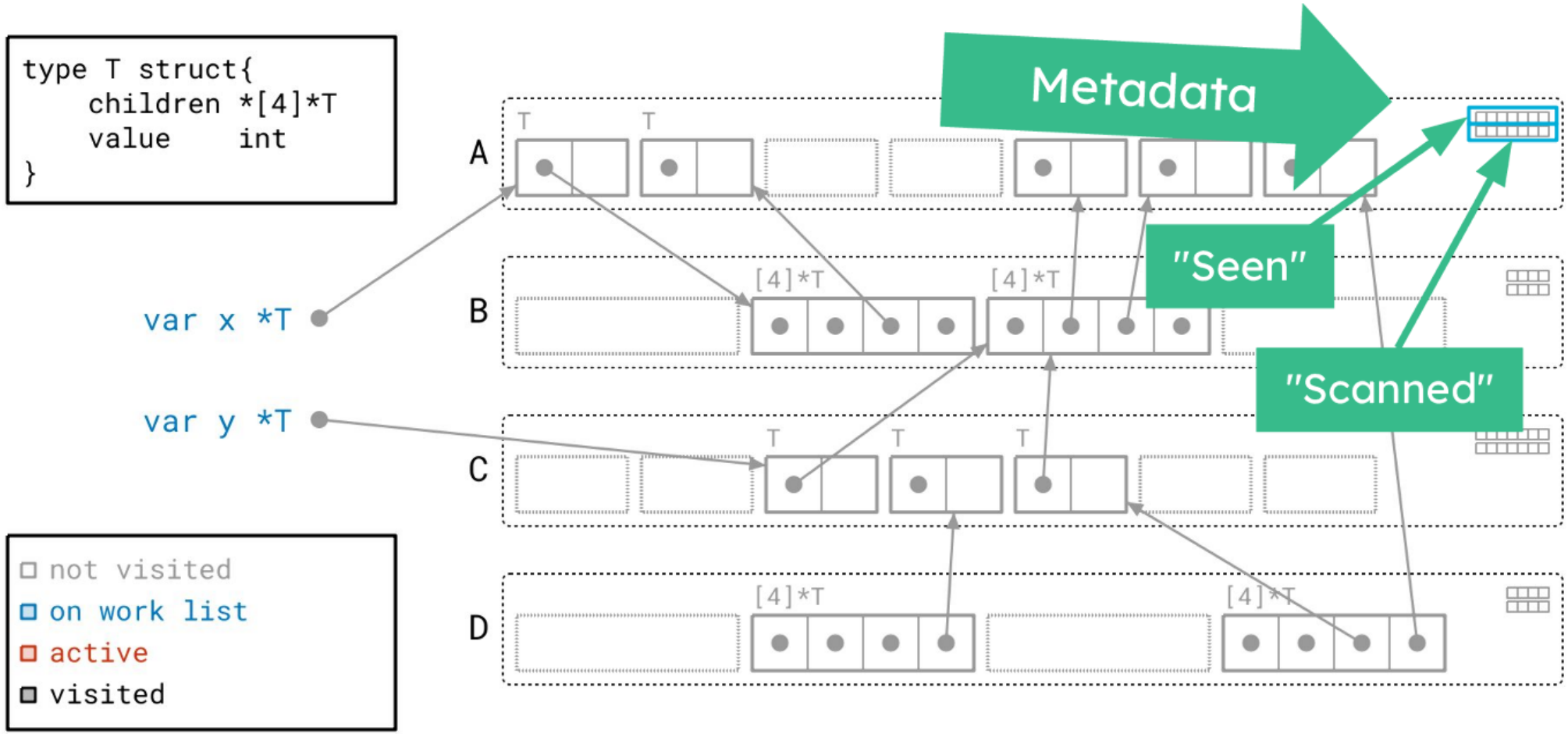
# GC classic phases

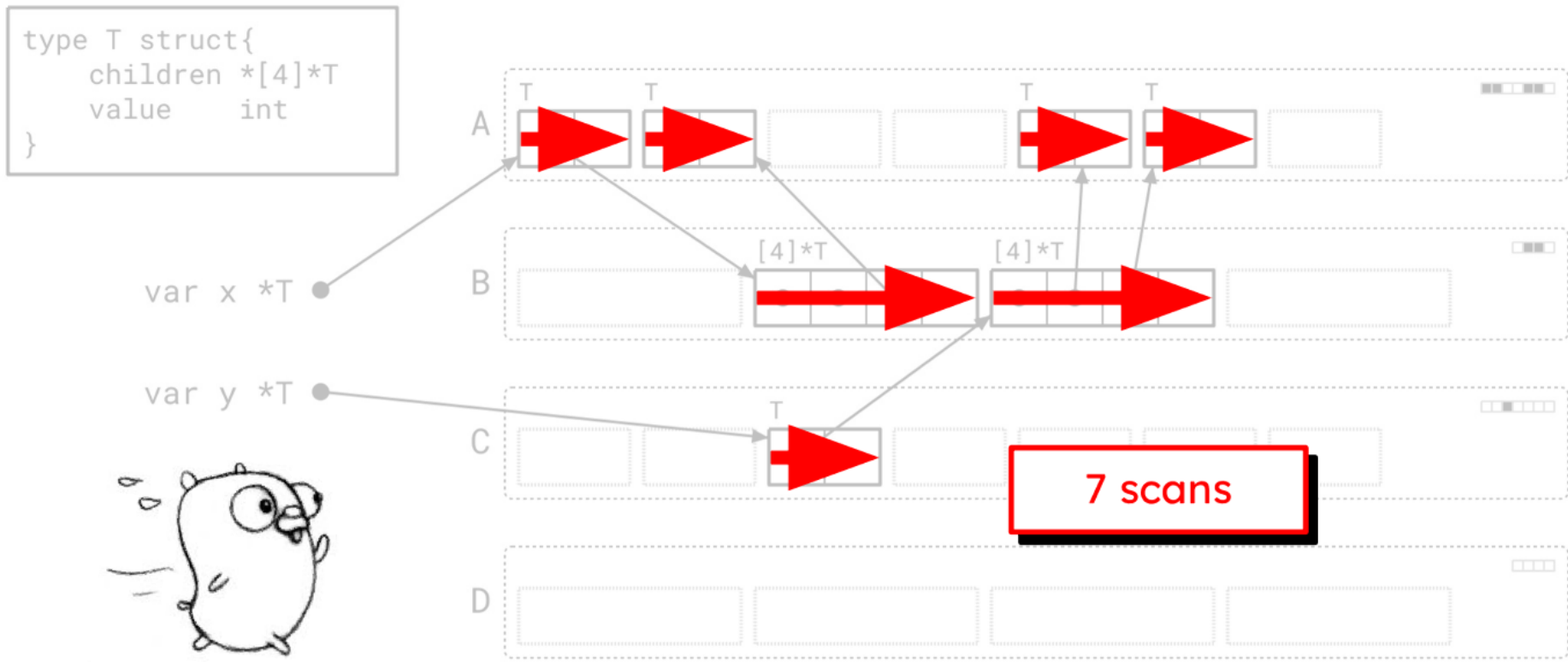
- **Sweep termination** — completes any pending sweep work from previous cycles.
- **Mark (concurrent)** — the marking phase (determining which objects are reachable) runs concurrently with the program, thanks to mechanisms that allow execution to continue with very short pauses.
- **Mark termination (STW)** — a short stop-the-world pause to finalize marking.
- **Sweep (concurrent or lazy)** — unmarked (white) objects are considered garbage and memory is freed in the background while the application continues running.
- It uses a **write barrier**: when the program (the mutator) modifies pointers during marking, the write barrier ensures the GC does not lose references to live objects.
- The GC is automatically triggered when the heap grows beyond a certain threshold; the default threshold is controlled by the GOGC environment variable.

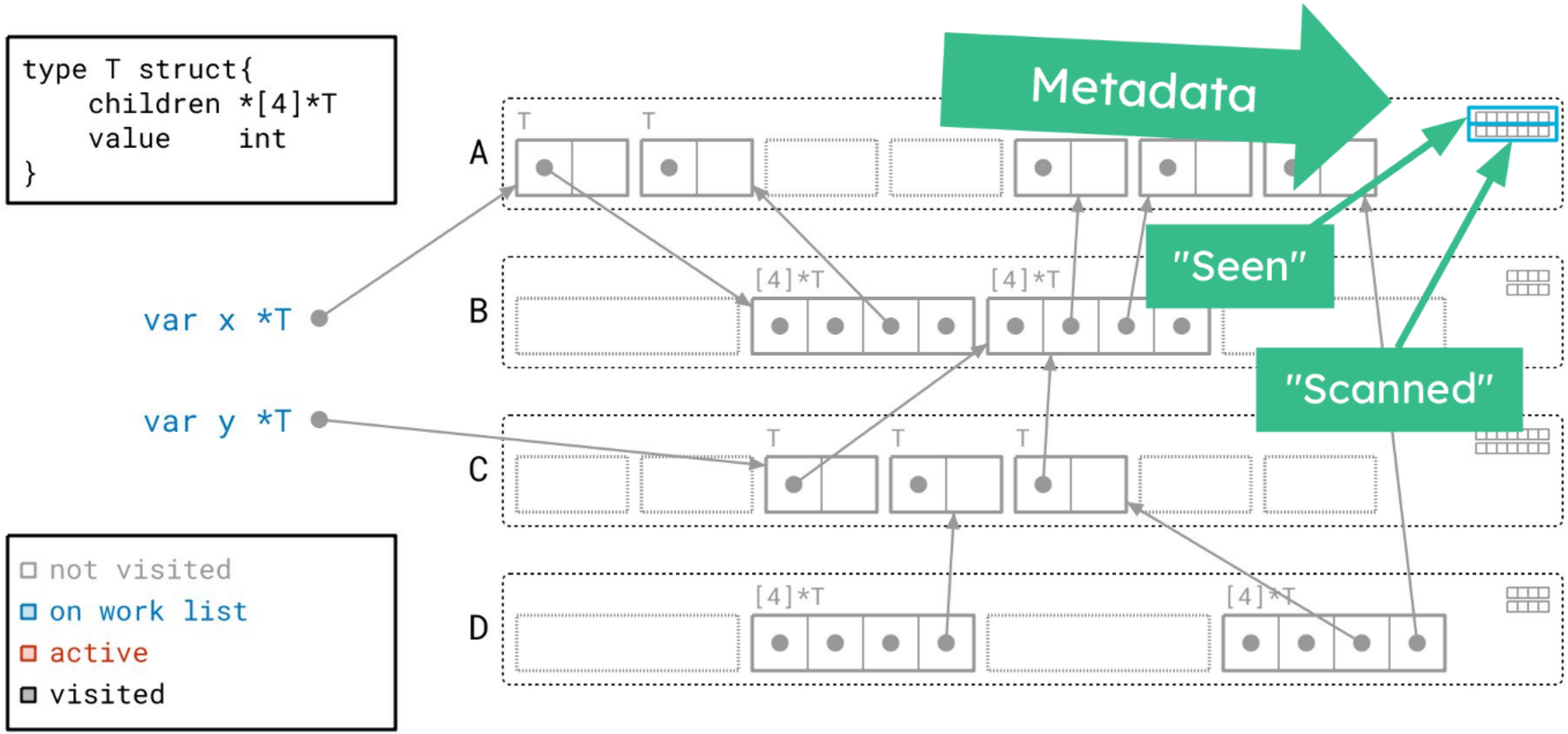
# Green Tea GC

- Go 1.25 includes a new experimental garbage collector called Green Tea, available by setting GOEXPERIMENT=greenteagc at build time. Many workloads spend around 10% less time in the garbage collector, but some workloads see a reduction of up to 40%!
- It’s production-ready and already in use at Google, so we encourage you to try it out. We know some workloads don’t benefit as much, or even at all, so your feedback is crucial to helping us move forward. Based on the data we have now, we plan to make it the default in Go 1.26.

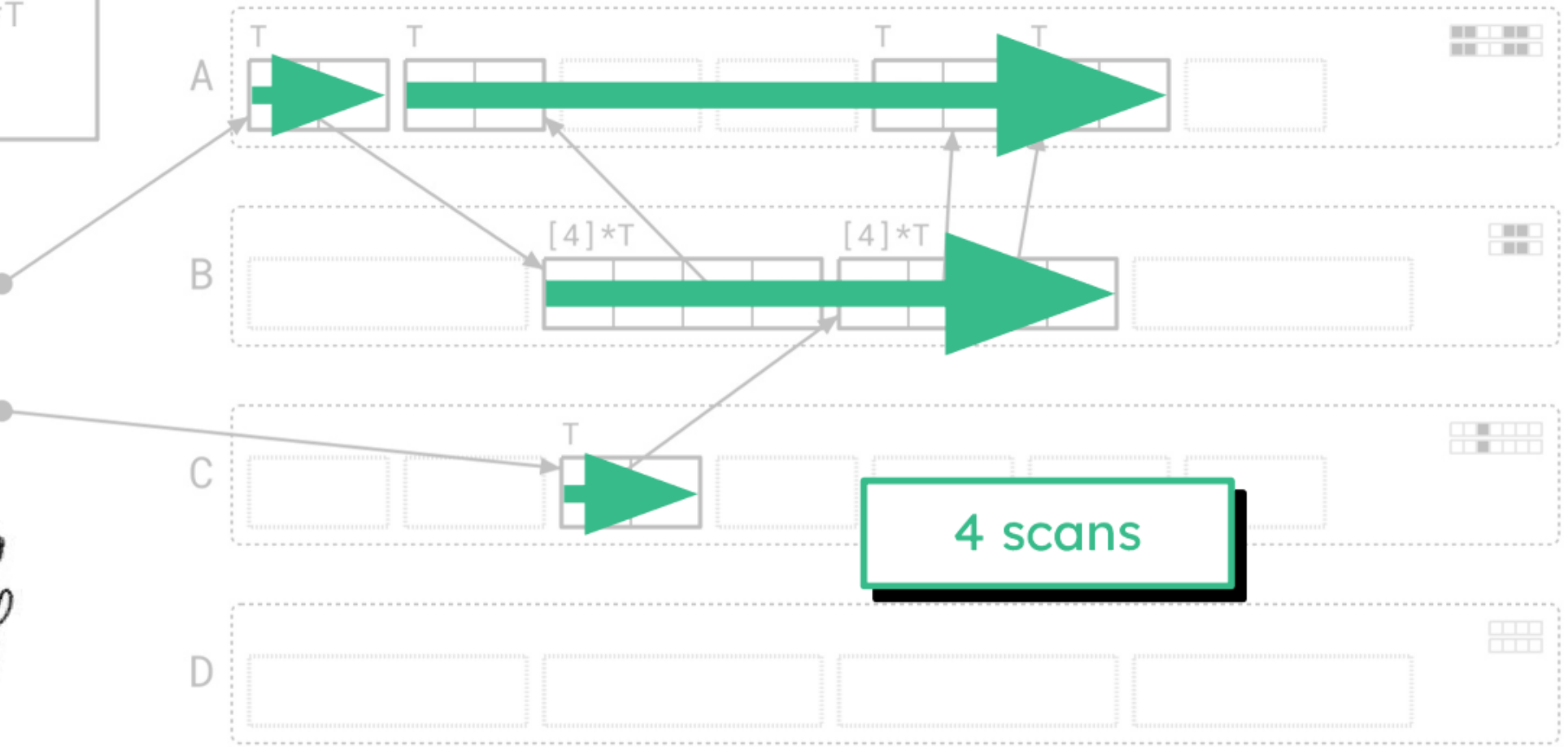
https://go.dev/blog/greenteagc


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
not visited
on work list
active
visited
A
B
C
D
T
[4]*T
Metadata
"Seen"
"Scanned"


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
T
T
T
[4]*T
[4]*T
T
7 scans


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
not visited
on work list
active
visited
A
B
C
D
T
[4]*T
Metadata
"Seen"
"Scanned"

```
type T struct{
    children *[4]*T
    value    int
}
```



# Benefit Green Tea GC

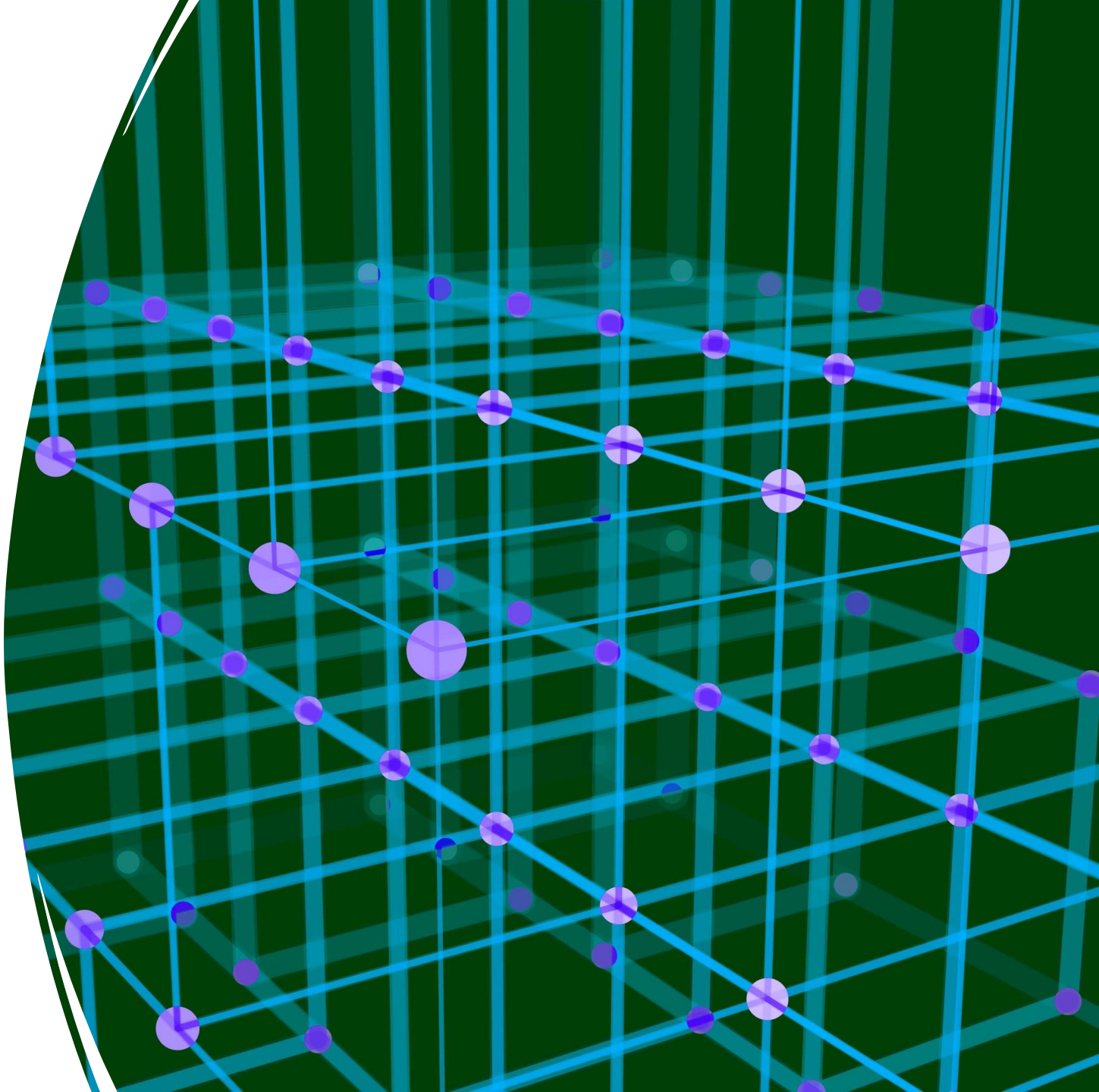
- reductions in garbage collection CPU costs between 10% and 40%
- Experiment on vector enhancements suggest this will net an additional 10% GC CPU reduction.

# MEMORY MANAGEMENT:HEAP

- Heap memory is used to allocate memory dynamically.
- The management of heap memory is the responsibility of Go's garbage collector.
- Memory is allocated on the heap using the new or make keyword to create new objects or to allocate.

| Feature | Generational GC | Go GC |
|---|---|---|
| **Structure** | Divides memory into generations (young, old) | Treats the heap as a single memory space |
| **Young Object Allocation** | Young objects are collected frequently | Temporary objects may be allocated on the stack |
| **Temporal Optimization** | Minor GC (young) is very fast, Major GC less frequent | Concurrent GC with minimal pauses |
| **Object Promotion** | Surviving young objects are promoted to the old generation | No explicit promotion |
| **Pause Time** | Reduced pause for minor collections | Minimal pause due to incremental collection |
| **Profiling and Optimization** | Specific profiling for generations | Adaptive GC, optimized based on workload |

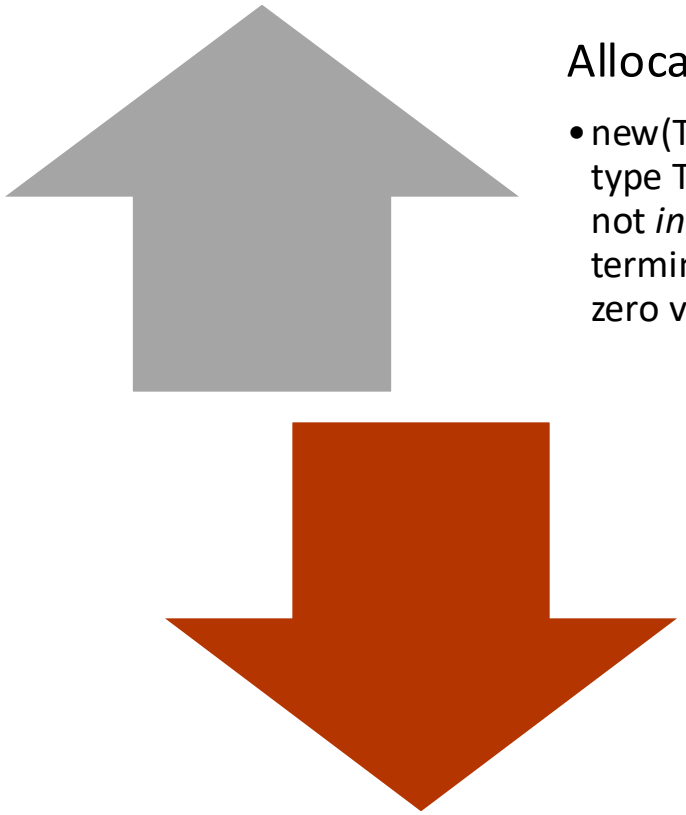
# Built in function



| Function | Purpose | Return Type | Example |
| --- | --- | --- | --- |
| `new` | Allocates memory for a type, returns pointer to zero-value | `*T` | `p := new(int); *p = 10` |
| `panic` | Stops normal execution, triggers runtime error | `void` | `panic("error")` |
| `recover` | Recovers from a panic within a deferred function | `interface{}` | `r := recover()` |
| `complex` | Creates a complex number from real and imaginary parts | `complex128` or `complex64` | `c := complex(2,3)` |
| `real` | Returns the real part of a complex number | float64/float32 | `real(complex(2,3)) // 2` |
| `imag` | Returns the imaginary part of a complex number | float64/float32 | `imag(complex(2,3)) // 3` |
| Type conversions | Converts values between types (int, string, float64…) | depends on conversion | `string(65) // "A"` |

| Function | Purpose | Return Type | Example |
| --- | --- | --- | --- |
| `len` | Returns length of array, slice, map, string, or channel | `int` | `len([]int{1,2,3}) // 3` |
| `cap` | Returns capacity of array, slice, or channel | `int` | `cap(make([]int, 3, 10)) // 10` |
| `append` | Appends elements to a slice | `[]T` | `s := []int{1,2}; s = append(s, 3) // [1 2 3]` |
| `copy` | Copies elements from one slice to another | `int` (number of elements copied) | `copy(dst, src)` |
| `delete` | Deletes a key from a map | `void` | `delete(m, "key")` |
| `make` | Creates slices, maps, or channels | slice/map/chan | `s := make([]int, 5)` |
| `close` | Closes a channel | `void` | `ch := make(chan int); close(ch)` |

# Memory management: heap allocation



## Allocation with new

- new(T) allocates zeroed storage for a new item of type T and returns its address, a value of type *T. Iit does not *initialize* the memory, it only *zeros* it. In Go terminology, it returns a pointer to a newly allocated zero value of type T.

## Allocation with make

- make(T, *args*) creates slices, maps, and channels only, and it returns an *initialized* (not *zeroed*) value of type T (not *T). These three types represent, under the covers, references to data structures that must be initialized before use.

# Pointer

# Pointer

Declaring pointers
*T is the type of the pointer variable which points to a value of type T.

```
func main() {
    b := 255
    var a *int = &b
    fmt.Printf("Type of a is %T\n", a)
    fmt.Println("address of b is", a)
}
```

The & operator is used to get the address of a variable.
Dereferencing a pointer means accessing the value of the variable which the pointer points to. *a

The zero value of a pointer is nil.

# Returning a pointer from a function

```
func hello() *int {
    i := 5
    return &i
}
func main() {
    d := hello()
    fmt.Println("Value of d", *d)
}
```

**In Go, the compiler does a escape analysis and allocates i on the heap as the address escapes the local scope**

# New

```
var p int
var v *int
v = &p
*v = 12
 fmt.Println(*v)
```

12

```
var v *int
*v = 8
fmt.Println(*v)
```

panic: runtime error: invalid memory address or nil pointer dereference
[signal SIGSEGV: segmentation violation code=0x1 addr=0x0 pc=0x487193]

```
var v *int
v = new(int)
*v = 8
fmt.Println(*v)
```

8

```
var v *int
// fmt.Println(*v)  //error
fmt.Println(v)
v = new(int)
fmt.Println(*v)
fmt.Println(v)
```

<nil>
0
0xc000016090

# pointer arithmetic

- Pointer arithmetic is allowed, but with some restrictions.
- Pointer arithmetic operators, such as + and -, can be used to navigate through elements of an array or move within allocated memory.

Type-based arithmetic: Pointer arithmetic in Go is based on the type of the elements being pointed to.

Accessing array bounds: It is the programmer's responsibility to ensure that array bounds are not exceeded. Pointer arithmetic can lead to accessing invalid memory locations if not done carefully.

```
package main

import "fmt"

func main() {
    // Creazione di un array di interi
    numbers := [...]int{1, 2, 3, 4, 5}

    // Ottenere un puntatore al primo elemento dell'array
    ptr := &numbers[0]

    // Utilizzo dell'aritmetica dei puntatori per accedere agli
elementi successivi
    for i := 0; i < len(numbers); i++ {
        fmt.Printf("Elemento %d: %d\n", i, *ptr)
        ptr = ptr + 1 // Incremento del puntatore di sizeof(int) byte
    }
}
```

# Array & slice

# Array: definition & initialitation

Differences between the ways arrays work in Go and C.

1. Arrays are values. Assigning one array to another copies all the elements.
2. if you pass an array to a function, it will receive a *copy* of the array, not a pointer to it.
3. the size of an array is part of its type.
4. The types [10]int and [20]int are distinct.

```
var vector *[N]T
```

```
a := [4]float64
```

```
a := [...]float64{67.7, 89.8, 21, 78}
```

```
x := [5]int{1: 10, 3: 30}
```

# Copy Array

```
package main

import "fmt"

func main() {

	strArray1 := [3]string{"Japan", "Australia", "Germany"}
	strArray2 := strArray1  // data is passed by value
	strArray3 := &strArray1 // data is passed by refrence

	fmt.Printf("strArray1: %v\n", strArray1)
	fmt.Printf("strArray2: %v\n", strArray2)

	strArray1[0] = "Canada"

	fmt.Printf("strArray1: %v\n", strArray1)
	fmt.Printf("strArray2: %v\n", strArray2)
	fmt.Printf("*strArray3: %v\n", *strArray3)
}
```

# Array

**C**

int *v

int v[4]

**Go**

# Arrays

```
func f(a [3]int) { fmt.Println(a) }
func fp(a *[3]int) { fmt.Println(a) }
func main() {
 var ar [3] int
 f(ar)         // passes a copy of ar
 fp(&ar) // passes a pointer to ar
}
```

Output (Print and friends know about arrays):
[0 0 0]
&[0 0 0]

# Arrays

var ar [3]int  //declares ar to be an array of 3 integers, initially all set to zero.

Size is part of the type.

Built-in function len reports size:

```
len(ar)  ==  3
```

Arrays are values, not implicit pointers as in C. You can take an array's address, yielding a pointer to the array

# Iterating arrays using range

```
for indice, valore := range collezione {
    // codice da eseguire
}
```

# Iterating arrays using range

```
package main

import "fmt"

func main() {
    a := [...]float64{67.7, 89.8, 21, 78}
   sum := float64(0)
    for i, v := range a {//range returns both the index and value
        fmt.Printf("%d the element of a is %.2f\n", i, v)
        sum += v
    }

    fmt.Println("\nsum of all elements of a",sum)
}

Func f1(int[]
```

```
for i ,value:= range numeri {
      value = value *= 2 // Non Modifica diretta dell'elemento }
```

```
for i ,value:= range numeri {
      numeri[i] *= 2 // Modifica diretta dell'elemento }
```

# Range on array

```
package main
import "fmt"
func main() {
	intArray := [5]int{10, 20, 30, 40, 50}
	fmt.Println("\n--------------Example 1-------------------\n")
	for i := 0; i < len(intArray); i++ {
		fmt.Println(intArray[i])
	}
	fmt.Println("\n--------------Example 2-------------------\n")
	for index, element := range intArray {
		fmt.Println(index, "=>", element)
	}
	fmt.Println("\n--------------Example 3-------------------\n")
	for _, value := range intArray {
		fmt.Println(value)
	}
	j := 0
	fmt.Println("\n--------------Example 4-------------------\n")
	for range intArray {
		fmt.Println(intArray[j])
		j++
	}
}
```

# Multidimensional arrays

```
var array [righe][colonne]tipo
```

```
matrix := [2][3]int{
    {1, 2, 3},
    {4, 5, 6},
}
```

```
package main

import "fmt"

func main() {
    matrix := [2][3]int{
        {1, 2, 3},
        {4, 5, 6},
    }

    // Iterazione su righe e colonne
    for i := 0; i < len(matrix); i++ {
        for j := 0; j < len(matrix[i]); j++ {
            fmt.Printf("Elemento [%d][%d]: %d\n", i, j, matrix[i][j])
        }
    }
}
```

# Multidimensional arrays

```
package main
import (
	"fmt"
)
func printarray(a [3][2]int) {
	for _, v1 := range a {
		for _, v2 := range v1 {
			fmt.Printf("%d. ",v2)
		}
		fmt.Printf("\n")
	}
}

func main() {
	a := [3][2]int {{1,2},{3,4},{5,6},}
	printarray(a)
	a[0][0] = 234;
	printarray(a)
}
```

# len

- len (collection)

```
package main

import "fmt"

// Funzione che accetta un array multidimensionale 2x3
func stampaMatrice(m [2][3]int) {
    for i := 0; i < len(m); i++ {
        for j := 0; j < len(m[i]); j++ {
            fmt.Printf("%d ", m[i][j])
        }
        fmt.Println()
    }
}

func main() {
    matrix := [2][3]int{
        {1, 2, 3},
        {4, 5, 6},
    }

    // Chiamata della funzione passando l'array
    stampaMatrice(matrix)
}
```

# Check if Element Exists

```
package main
import (
	"fmt"
	"reflect"
)
func main() {
	strArray := [5]string{"India", "Canada", "Japan", "Germany", "Italy"}
	fmt.Println(itemExists(strArray, "Canada"))
	fmt.Println(itemExists(strArray, "Africa"))
}
func itemExists(arrayType interface{}, item interface{}) bool {
	arr := reflect.ValueOf(arrayType)
	if arr.Kind() != reflect.Array {
		panic("Invalid data-type")
	}
	for i := 0; i < arr.Len(); i++ {
		if arr.Index(i).Interface() == item {
			return true
		}
	}
	return false
}
```

# Pointers to array literals

You can take the address of an array literal to get a pointer to a newly created instance:

```
func fp(a *[3]int) { fmt.Println(a) }
func main() {
  for  i  :=  0;  i  <  3;  i++  {
        fp(&[3]int{i,  i*i,  i*i*i})
  }
}
```

Output:
&[0 0 0]
&[1 1 1]
&[2 4 8]

# Array on heap

```
var a [5]int
fmt.Printf("a: %p %#v \n", &a, a)
av := new([5]int)
fmt.Printf("av: %p %#v \n", &av, av)
(*av)[1] = 8
fmt.Printf("av: %p %#v \n", &av, av)
```

```
a: 0xc00008c030 [5]int{0, 0, 0, 0, 0}
av: 0xc00009a020 &[5]int{0, 0, 0, 0, 0}
av: 0xc00009a020 &[5]int{0, 8, 0, 0, 0}
```

# Slices

A slice is a reference to a section of an array.  Slices are used much more often than plain arrays.

A slice is very cheap.

A slice type looks like an array type without a size:

```
var a []int
```

Create a slice by "slicing" an array or slice:

```
a  =  ar[7:9].     When slicing, first index defaults to 0:
a =   ar[:n]    //means the same as ar[0:n].
ar[n:]     // means the same as ar[n:len(ar)].
ar[:]              //means the same as ar[0:len(ar)].
```

# Length and capacity of a slice

- The length of the slice is the number of elements in the slice.
- **The capacity of the slice is the number of elements in the underlying array starting from the index from which the slice is created.**

```
package main

import (
"fmt"
)

func main() {
        array := [...]int{1,2,3,4,5,6,7,8,9,10}
        arrayslice := array[1:3]
        fmt.Printf("length of slice %d capacity %d»,
                                        len(arrayslice), cap(arrayslice))
}
```

# Declare Slice using Make

```
package main

import (
	"fmt"
	"reflect"
)

func main() {
	var intSlice = make([]int, 10)	// when length and capacity is same
	var strSlice = make([]string, 10, 20) // when length and capacity is different

	fmt.Printf("intSlice \tLen: %v \tCap: %v\n", len(intSlice), cap(intSlice))
	fmt.Println(reflect.ValueOf(intSlice).Kind())

	fmt.Printf("strSlice \tLen: %v \tCap: %v\n", len(strSlice), cap(strSlice))
	fmt.Println(reflect.ValueOf(strSlice).Kind())
}
```

# Initialize Slice with values using a Slice Literal

```
package main

import "fmt"

func main() {
	var intSlice = []int{10, 20, 30, 40}
	var strSlice = []string{"India", "Canada", "Japan"}

	fmt.Printf("intSlice \tLen: %v \tCap: %v\n", len(intSlice), cap(intSlice))
	fmt.Printf("strSlice \tLen: %v \tCap: %v\n", len(strSlice), cap(strSlice))
}
```

# Declare Slice using new Keyword

```
package main

import (
	"fmt"
	"reflect"
)

func main() {
	var intSlice = new([50]int)[0:10]

	fmt.Println(reflect.ValueOf(intSlice).Kind())
	fmt.Printf("intSlice \tLen: %v \tCap: %v\n", len(intSlice), cap(intSlice))
	fmt.Println(intSlice)
}
```

```go
package main

import "fmt"

func main() {
    // Dichiarazione di uno slice vuoto
    var mySlice []int

    // Aggiunta di elementi allo slice
    mySlice = append(mySlice, 1)
    mySlice = append(mySlice, 2, 3, 4, 5)

    // Stampa dello slice
    fmt.Println(mySlice) // Output: [1 2 3 4 5]
}
```

# Add Items

```
package main

import "fmt"

func main() {
	a := make([]int, 2, 5)
	a[0] = 10
	a[1] = 20
	fmt.Println("Slice A:", a)
	fmt.Printf("Length is %d Capacity is %d\n", len(a), cap(a))

	a = append(a, 30, 40, 50, 60, 70, 80, 90)
	fmt.Println("Slice A after appending data:", a)
	fmt.Printf("Length is %d Capacity is %d\n", len(a), cap(a))
}
```

# Resizing a slice

Slices can be used like growable arrays. Allocate one using make with two numbers - length and capacity - and reslice as it grows:

```
var sl = make([]int, 0, 100)        //len 0, cap 100
func appendToSlice(i int, sl []int) []int {
  if len(sl) == cap(sl) { error(...) }
  n := len(sl)
  sl = sl[0:n+1]          // extend length by 1
  sl[n] = i
  return sl
}
```

Thus sl's length is always the number of elements, but it grows as needed.
This style is cheap and idiomatic in Go.

# Remove Item from Slice

```
package main

import "fmt"

func main() {
	var strSlice = []string{"India", "Canada", "Japan",
"Germany", "Italy"}
	fmt.Println(strSlice)

	strSlice = RemoveIndex(strSlice, 3)
	fmt.Println(strSlice)
}

func RemoveIndex(s []string, index int) []string {
	return append(s[:index], s[index+1:]...)
}
```

# Copy a Slice: function copy

```
package main

import "fmt"

func main() {
	a := []int{5, 6, 7} // Create a smaller slice
	fmt.Printf("[Slice:A] Length is %d Capacity is %d\n", len(a),
cap(a))

	b := make([]int, 5, 10) // Create a bigger slice
	copy(b, a)		// Copy function
	fmt.Printf("[Slice:B] Length is %d Capacity is %d\n", len(b),
cap(b))

	fmt.Println("Slice B after copying:", b)
	b[3] = 8
	b[4] = 9
	fmt.Println("Slice B after adding elements:", b)
}
```

# Append a slice to an existing slice

```
package main

import "fmt"

func main() {
	var slice1 = []string{"india", "japan", "canada"}
	var slice2 = []string{"australia", "russia"}

	slice2 = append(slice2, slice1...)
}
```

# Check if Item Exists

```
package main
import (
	"fmt"
	"reflect"
)
func main() {
	var strSlice = []string{"India", "Canada", "Japan", "Germany", "Italy"}
	fmt.Println(itemExists(strSlice, "Canada"))
	fmt.Println(itemExists(strSlice, "Africa"))
}
func itemExists(slice interface{}, item interface{}) bool {
	s := reflect.ValueOf(slice)
	if s.Kind() != reflect.Slice {
		panic("Invalid data-type")
	}
	for i := 0; i < s.Len(); i++ {
		if s.Index(i).Interface() == item {
			return true
		}
	}

	return false
}
```

# Array, slice & variadic functions

**Examples and understanding how variadic functions work**

**The way variadic functions work is by converting the variable number of arguments to a slice of the type of the variadic parameter.**

The find function expects a variadic int argument.
Hence these three arguments will be converted by the compiler to a slice of type int
[]int{89, 90, 95}
and then it will be passed to the find function.

```
import (
   "fmt"
)

func find(num int, nums ...int) {
   fmt.Printf("type of nums is %T\n", nums)
   found := false
   for i, v := range nums {
      if v == num {
         fmt.Println(num, "found at index", i, "in", nums)
         found = true
      }
   }
   if !found {
      fmt.Println(num, "not found in ", nums)
   }
   fmt.Printf("\n")
}
func main() {
   find(89, 89, 90, 95)
   find(45, 56, 67, 45, 90, 109)
   find(78, 38, 56, 98)
   find(87)
}
```

```go
package main

import "fmt"

// Funzione variadica che accetta un numero variabile di interi
func sum(numbers ...int) int {
    result := 0
    for _, n := range numbers {
        result += n
    }
    return result
}

// Funzione variadica che accetta un numero variabile di argomenti di diversi tipi
func printValues(values ...interface{}) {
    for _, v := range values {
        fmt.Println(v)
    }
}

func main() {
    // Utilizzo della funzione variadica sum
    total := sum(1, 2, 3, 4, 5)
    fmt.Println("Sum:", total)

    // Utilizzo della funzione variadica printValues
    printValues(1, "hello", 3.14, true)
}
```

# Slice arguments vs Variadic arguments

func find(num int, nums ...int)

func find(num int, nums []int)

The following of the advantages of using variadic arguments instead of slices.

- There is no need to create a slice during each function call.
- There is no need to create  an empty slice just to satisfy the signature of the find function.
- The program with variadic functions is more readable than the once with slices

**Append is a variadic function**

**To pass a slice to a variadic function we can use a syntactic sugar. You have to suffix the slice with ellipsis ... If that is done, the slice is directly passed to the functi without a new slice being created.**

```
nums := []int{89, 90, 95}
find(89, nums...)
```

# NEW AND MAKE TO SLICE ON HEAP

```
var a *[]int
    fmt.Printf("a: %p %#v \n", &a, a)
av := new([]int)
    fmt.Printf("av: %p %#v \n", &av, av)
(*av)[0] = 8
    fmt.Printf("av: %p %#v \n", &av, av)
```

```
a: 0xc00009a018 (*[]int)(nil)
av: 0xc00009a028 &[]int(nil)
panic: runtime error: index out of range
```

```
av := make([]int, 5)
    fmt.Printf("av: %p %#v \n", &av, av)
av[0] = 2
    fmt.Printf("av: %p %#v \n", &av, av)
```

```
av: 0xc00000c060 []int{0, 0, 0, 0, 0}
av: 0xc00000c060 []int{2, 0, 0, 0, 0}
```

# MORE Data type

Struct, Map, Function, methods, interface, channel

# MAP

# Maps

Maps are a convenient and powerful built-in data structure that associate values of one type (the *key*) with values of another type (the *element* or *value*)

key can be of any type for which the equality operator is defined

```
var m map[string]float64
```

```
attended := map[string]bool{ "Ann": true, "Joe": true, ... }
```

This declares a map indexed with key type string and value type float64. It is analogous to the C++ type *map<string,float64>>

The comparison operators == and != must be fully defined for operands of the key type; thus the key type must not be a function, map, or slice.
If the key type is an interface type, these comparison operators must be defined for the dynamic key values; failure will cause a run-time panic.

# Map initialization

```
package main

import "fmt"

var employee = map[string]int{"Mark": 10, "Sandy": 20}

func main() {
	fmt.Println(employee)
}
```

Empty Map declaration

Example

```
package main

import "fmt"

func main() {
	var employee = map[string]int{}
	fmt.Println(employee)	// map[]
	fmt.Printf("%T\n", employee) // map[string]int
}
```

# Map declaration using make function

```
package main

import "fmt"

func main() {
	var employee = make(map[string]int)
	employee["Mark"] = 10
	employee["Sandy"] = 20
	fmt.Println(employee)

	employeeList := make(map[string]int)
	employeeList["Mark"] = 10
	employeeList["Sandy"] = 20
	fmt.Println(employeeList)
}
```

# MAP

- Lenght::  number of map elements . For a map m, it can be discovered using the built-in function len and may change during execution.
- Elements may be added during execution using assignments and retrieved with index expressions; they may be removed with the delete built-in function.
- A new, empty map value is made using the built-in function make, which takes the map type and an optional capacity hint as arguments:

```
make(map[string]int)
make(map[string]int, 100)
```

- The initial capacity does not bound its size: maps grow to accommodate the number of items stored in them, with the exception of nil maps. A nil map is equivalent to an empty map except that no elements may be added.

# Map Length

```
package main

import "fmt"

func main() {
	var employee = make(map[string]int)
	employee["Mark"] = 10
	employee["Sandy"] = 20

	// Empty Map
	employeeList := make(map[string]int)

	fmt.Println(len(employee))     // 2
	fmt.Println(len(employeeList)) // 0
}
```

The len() function will return zero for an uninitialized map.

# Accessing Items

```
package main

import "fmt"

func main() {
	var employee = map[string]int{"Mark": 10, "Sandy": 20}

	fmt.Println(employee["Mark"])
}
```

# Adding Items

```
package main

import "fmt"

func main() {
	var employee = map[string]int{"Mark": 10, "Sandy": 20}
	fmt.Println(employee) // Initial Map

	employee["Rocky"] = 30 // Add element
	employee["Josef"] = 40

	fmt.Println(employee)
}
```

```go
package main

import (
    "fmt"
)

func main() {
    employeeSalary := map[string]int {
        "steve": 12000,
        "jamie": 15000,
    }
    employeeSalary["mike"] = 9000
    fmt.Println("employeeSalary map contents:", employeeSalary)
}
```

```go
func main() {
    employeeSalary := make(map[string]int)
    employeeSalary["steve"] = 12000
    employeeSalary["jamie"] = 15000
    employeeSalary["mike"] = 9000
    fmt.Println("employeeSalary map contents:", employeeSalary)
}
```

# Update Values

```
package main

import "fmt"

func main() {
	var employee = map[string]int{"Mark": 10, "Sandy": 20}
	fmt.Println(employee) // Initial Map

	employee["Mark"] = 50 // Edit item
	fmt.Println(employee)
}
```

# MAP

As with a slice, a map variable refers to nothing; you must put something in it before it can be used.

Three ways:

1) Literal: list of colon-separated key:value pairs

```
m = map[string]float64{“1”:1,  “pi”:3.1415}
```

2) Creation

```
m = make(map[string]float64)    // make not new
```

3) Assignment

```
var m1 map[string]float64
m1 = m          // m1 and m now refer to same map
```

# Map on heap

```
var m map[string]string
    fmt.Printf("m: %p %#v \n", &m, m)
    mv := new(map[string]string)
    fmt.Printf("mv: %p %#v \n", &mv, mv)
&map[string]string(nil)
    (*mv)["a"] = "a"
    fmt.Printf("mv: %p %#V \n ", & mv, mv)
```

```
m: 0xc00000e028 map[string]string(nil)
mv: 0xc00000e038 &map[string]string(nil)
panic: assignment to entry in nil map
```

```
var mv *map[string]string
fmt.Printf("mv: %p %#v \n", &mv, mv)
mv = new(map[string]string)
fmt.Printf("mv: %p %#v \n", &mv, mv)
(*mv) = make(map[string]string)
(*mv)["a"] = "a"
fmt.Printf("mv: %p %#v \n", &mv, mv)
```

```
mv: 0xc00000e028 (*map[string]string)(nil)
mv: 0xc00000e028 &map[string]string(nil)
mv: 0xc00000e028 &map[string]string{"a":"a"}
```

```
mv := make(map[string]string)
fmt.Printf("mv: %p %#v \n", &mv, mv)
mv["m"] = "m"
fmt.Printf("mv: %p %#v \n", &mv, mv)
```

```
mv: 0xc00009a018 map[string]string{}
mv: 0xc00009a018 map[string]string{"m":"m"}
```

# Indexing a map

```
m  = map[string]float64{“1”:1,  “pi”:3.1415}
```

Access an element as a value; if not present, get

zero value for the map's value type:

```
One :=  m[“1”]
zero := m["not present"] // Sets zero to 0.0.
```

Set an element (setting twice updates value for key)

```
m[“2”]  =  2
m["2"] = 3   // mess with their heads
```

# Testing existence

```
package main

import (
   "fmt"
)

func main() {
   employeeSalary := map[string]int{
      "steve": 12000,
      "jamie": 15000,
   }
   newEmp := "joe"
   value, ok := employeeSalary[newEmp]
   if ok == true {
      fmt.Println("Salary of", newEmp, "is", value)
      return
   }
   fmt.Println(newEmp, "not found")

}
```

# Deleting

func delete(m map[Type]Type1, key Type)

```
func main() {
    employeeSalary := map[string]int{
        "steve": 12000,
        "jamie": 15000,
        "mike": 9000,
    }
    fmt.Println("map before deletion", employeeSalary)
    delete(employeeSalary, "steve")
    fmt.Println("map after deletion", employeeSalary)

}
```

```
package main

import "fmt"

func main() {
	var employee = make(map[string]int)
	employee["Mark"] = 10
	employee["Sandy"] = 20
	employee["Rocky"] = 30
	employee["Josef"] = 40

	fmt.Println(employee)

	delete(employee, "Mark")
	fmt.Println(employee)
}
```

# For and range

The for loop has a special syntax for iterating over arrays, slices, maps

```
m  := map[string]float64{“1”:1.0,  “pi”:3.1415}
for key, value := range m {
        fmt.Printf("key %s, value %g\n", key, value)
}
```

With only one variable in the range, get the key:

```
for key = range m {
        fmt.Printf("key %s\n", key)
}
```

A for using range on a string loops over Unicode code

points, not bytes

```
s := "[\u00ff\u754c]"
for i, c := range s {
        fmt.Printf("%d:%q ", i, c) // %q for 'quoted'
```

# Truncate Map

```
package main

func main() {
	var employee = map[string]int{"Mark": 10, "Sandy": 20,
		"Rocky": 30, "Rajiv": 40, "Kate": 50}

	// Method - I
	for k := range employee {
		delete(employee, k)
	}

	// Method - II
	employee = make(map[string]int)
}
```

# Sort Map Keys

```
package main

import (
	"fmt"
	"sort"
)

func main() {
	unSortedMap := map[string]int{"India": 20, "Canada": 70,
"Germany": 15}

	keys := make([]string, 0, len(unSortedMap))

	for k := range unSortedMap {
		keys = append(keys, k)
	}
	sort.Strings(keys)

	for _, k := range keys {
		fmt.Println(k, unSortedMap[k])
	}
}
```

# Sort Map Values

```
package main
import (
	"fmt"
	"sort"
)
func main() {
	unSortedMap := map[string]int{"India": 20, "Canada": 70, "Germany": 15}

 // Int slice to store values of map.
	values := make([]int, 0, len(unSortedMap))

	for _, v := range unSortedMap {
		values = append(values, v)
	}
 // Sort slice values.
	sort.Ints(values)

 // Print values of sorted Slice.
	for _, v := range values {
		fmt.Println(v)
	}
}
```

# Merge Maps

```
package main

import "fmt"

func main() {
	first := map[string]int{"a": 1, "b": 2, "c": 3}
	second := map[string]int{"a": 1, "e": 5, "c": 3, "d": 4}

	for k, v := range second {
		first[k] = v
	}

	fmt.Println(first)
}
```

# String

# String

- A string type represents the set of string values. A string value is a (possibly empty) sequence of bytes.
- The number of bytes is called the length of the string and is never negative.
- Strings are **immutable**: once created, it is impossible to change the contents of a string.
- The predeclared string type is **string**; it is a defined type.
- The length of a string s can be discovered using the built-in function len.
- The length is a compile-time constant if the string is a constant.
- A string's bytes can be accessed by integer indices 0 through len(s)-1.
- It is illegal to take the address of such an element; if s[i] is the i'th byte of a string, &s[i] is invalid.

# String

A string is a slice of bytes in Go. Strings can be created by enclosing a set of characters inside double quotes " »

**Accessing individual bytes of a string**

```
func main() {
        name := "Hello World"
        fmt.Printf("String: %s\n", name)
        fmt.Printf(name[1:2])
        fmt.Printf("\n%c", name[1])
}
```

**func RuneCountInString(s string) (n int) returns the number of runes in it.**

**len(s) is used to find the number of bytes in the string and it doesn't return the string length**

```
import ("fmt"
        "unicode/utf8"
        )
```

```
func main() {
word1 := "Señor"
fmt.Printf("String: %c\n", word1[0])
fmt.Printf("Length: %d\n", utf8.RuneCountInString(word1))
fmt.Printf("Number of bytes: %d\n", len(word1))

fmt.Printf("\n")
word2 := "Pets"
fmt.Printf("String: %s\n", word2)
fmt.Printf("Length: %d\n", utf8.RuneCountInString(word2))
fmt.Printf("Number of bytes: %d\n", len(word2))
}
```

```
String: S
Length: 5
Number of bytes: 6

String: Pets
Length: 4
Number of bytes: 4
```

# String comparing

- Using operator ==

```
func main() {
str1 := "Go"
str2 := "Go111"
str3 :="Go111"
if str1 == str2 {
        fmt.Printf("equal")
        } else {fmt.Printf("not equal")}
if str2 == str3 {
        fmt.Printf("equal")
        } else {fmt.Printf("not equal")}

}
```

Output
not equal
equal

# Mutable string

| Approach | Mutable? | Efficient? | Notes |
| --- | --- | --- | --- |
| `string + string` | ✗ No | ✗ No | Creates copies |
| `[]byte` | ✓ Yes | ✓ Yes | Lower-level manipulation |
| `bytes.Buffer` | ✓ Yes | ✓ Yes | Great for byte-oriented data |
| **`strings.Builder`** | ✓ Yes | **Best for string building** | Purpose-built for strings |

# [] Byte

```
s := "ciao"
b := []byte(s)
```

```
b[0] = 'm'
```

"ciao" → [99 105 97 111]

```
b = append(b, '!')
```

```
s2 := string(b)
fmt.Println(s2)
```

| Operation | `string` | `[]byte` |
| --- | --- | --- |
| Mutable | ✗ No | ✓ Yes |
| Access characters | Read-only | Read & write |
| Efficient concatenation | No | Yes, with `append` |
| Convert **from** string | — | `[]byte(s)` |
| Convert **to** string | — | `string(b)` |

# bytes.Buffer

```
package main

import (
    "bytes"
    "fmt"
)

func main() {
    var buf bytes.Buffer

    buf.WriteString("Hello")
    buf.WriteByte(' ')
    buf.WriteString("Go!")

    fmt.Println(buf.String())   // Output: Hello Go!
}
```

# String builder

```
import "strings"
```

```
var sb strings.Builder

sb.WriteString("Ciao ")
sb.WriteString("mondo!")
fmt.Println(sb.String())
```

```
var sb strings.Builder
sb.Grow(50)  // Prealloca 50 byte
```

# Methods

| Method | Description |
| --- | --- |
| `WriteString(s string)` | Writes a string |
| `Write([]byte)` | Writes a byte slice |
| `WriteRune(r rune)` | Writes a Unicode character |
| `WriteByte(b byte)` | Writes a single byte |
| `String()` | Returns the final string |
| `Reset()` | Clears the builder (without freeing memory) |

# Struct

```
type identifier struct{
  field1 data_type
  field2 data_type
  field3 data_type
}
```

```
type Person struct {
    Name string
    Age  int
}
```

# Creating instances

```
p := Person{
    Name:
"Alice",
    Age:  30,
}
```

```
p := Person{"Alice",
30}
```

```
p := &Person{
    Name: "Bob",
    Age:  25,
}.
```

# Field access

```
p := Person{Name: "Alice", Age: 30}
fmt.Println(p.Name) // "Alice"

p.Age = 31 // modifica campo
fmt.Println(p.Age) // 31
```

```
p := &Person{
    Name: "Bob",
    Age:  25,
}
fmt.Println(p.Name) // accesso con .
```

# Creating Instances of Struct Types

```
type rectangle struct {
        length  int
        breadth int
        color   string

        geometry struct {
                area      int
                perimeter int
        }
}
                rect.length = 10
                        rect.breadth = 20
                        rect.color = "Green"

                        rect.geometry.area = rect.length * rect.breadth
                        rect.geometry.perimeter = 2 * (rect.length + rect.breadth)
```

# Creating a Struct Instance Using a Struct Literal

```
type rectangle struct {
        length  int
        breadth int
        color   string
}

func main() {
        var rect1 = rectangle{10, 20, "Green"}
        fmt.Println(rect1)

        var rect2 = rectangle{length: 10, color: "Green"} // breadth value skipped
        fmt.Println(rect2)

        rect3 := rectangle{10, 20, "Green"}
        fmt.Println(rect3)

        rect4 := rectangle{length: 10, breadth: 20, color: "Green"}
        fmt.Println(rect4)

        rect5 := rectangle{breadth: 20, color: "Green"} // length value skipped
        fmt.Println(rect5)
```

# Struct Instantiation Using new Keyword

```
package main

import "fmt"

type rectangle struct {
	length  int
	breadth int
	color   string
}

func main() {
	rect1 := new(rectangle) // rect1 is a pointer to an instance of rectangle
	rect1.length = 10
	rect1.breadth = 20
	rect1.color = "Green"
	fmt.Println(rect1)

	var rect2 = new(rectangle) // rect2 is an instance of rectangle
	rect2.length = 10
	rect2.color = "Red"
	fmt.Println(rect2)
}
```

# Anonymous struct

```
person := struct {
    Name string
    Age  int
}{
    Name: "Alice",
    Age:  30,
}
```

```
people := []struct {
    Name string
    Age  int
}{
    {Name: "Alice", Age: 30},
    {Name: "Bob", Age: 25},
}

fmt.Println(people[0].Name) // "Alice"
```

# Structs are values

Structs are values and new(StructType) returns a pointer to a zero value (memory all zeros).

```
type Point struct {
        x, y float64
}
var p Point
p.x = 7
p.y = 23.4
var pp *Point = new(Point)
*pp = p
pp.x = Pi // sugar for (*pp).x
```

There is no -> notation for structure pointers. Go provides the indirection for you.

# Making structs

Structs are values so you can make a zero ed one just by declaring it.

You can also allocate one with new.

```
var p Point // zeroed value
pp := new(Point) // idiomatic allocation
Struct literals have the expected syntax.
p = Point{7.2, 8.4}
p = Point{y:8.4, x:7.2}
pp = &Point{7.2, 8.4} // idiomatic
pp = &Point{} // also idiomatic; == new(Point)
```

As with arrays, taking the address of a struct literal gives the address of a newly created value.

These examples are constructors.

# Exporting types and fields

The fields of a struct must start with an Uppercase letter to be visible outside the package.

Private type and fields:

```
type point struct { x, y float64 }
```

Exported type and fields:

```
type Point struct { X, Y float64 }
```

Exported type with mix of fields:

```
type Point struct {
        X, Y float64        // exported
        name string         // not exported
}
```

You may even have a private type with exported fields.

# Anonymous fields

Inside a struct, you can declare fields, such as another struct, without giving a name for the field.

These are called anonymous fields and they act as if the inner struct is simply inserted or "embedded" into the outer.

This simple mechanism provides a way to derive some or all of your implementation from another type or types.

```
type A struct {
        ax, ay int
}
type B struct {
        A
        bx, by float64
}
```

B acts as if it has four fields, ax, ay, bx, and by. It's almost as if B is {ax, ay int; bx, by float64}. However, literals for B must be filled out in detail:

```
b := B{A{1, 2}, 3.0, 4.0}
fmt.Println(b.ax, b.ay, b.bx, b.by)
```

Prints 1 2 3 4

# Anonymous fields have type as name

But it's richer than simple interpolation of the fields:

B also has a field A. The anonymous field looks like

a field whose name is its type.

```
b := B{A{ 1, 2}, 3.0, 4.0}
fmt.Println(b.A)
```

Prints {1 2}. If A came from another package, the field would still be called A:

```
import "pkg"
type C struct { pkg.A }
...
c := C {pkg.A{1, 2}}
fmt.Println(c.A) // not c.pkg.A
```

# Conflict examples

```
type A struct { a int }
type B struct { a, b int }
type C struct { A; B }
var c C
```

- Using c.a is an error: is it c.A.a or c.B.a?

```
type D struct { B; b float64 }
var d D
```

- Using d.b is OK: it's the float64, not d.B.b
- Can get at the inner b by D.B.b.

# Conflicts and hiding

If there are two fields with the same name (possibly a type-derived name), these rules apply:

1. An outer name hides an inner name. This provides a way to override a field/method.
2. If the same name appears twice at the same level,

it is an error if the name is used by the program.

No rules to resolve the ambiguity; it must be fixed.

# Struct Comparinson

```
package main
import "fmt»

type name struct {
   firstName string
   lastName  string
}
func main() {
   name1 := name{
      firstName: "Steve",
      lastName:  "Jobs",
   }
   name2 := name{
      firstName: "Steve",
      lastName:  "Jobs",
   }
   if name1 == name2 {
      fmt.Println("name1 and name2 are equal")
   } else {
      fmt.Println("name1 and name2 are not equal")
   }
}
```

# Type function

# Function type

- A function type denotes the set of all functions with the same parameter and result types. The value of an uninitialized variable of function type is nil.

```
func() func(x int) int
func(a, _ int, z float32) bool
func(a, b int, z float32) (bool)
func(prefix string, values ...int)
func(a, b int, z float64, opt ...interface{}) (success bool) func(int,
int, float64) (float64, *[]int)
func(n int) func(p *T)
```

- Functions are values too. They can be passed around just like other values.
- Function values may be used as function arguments and return values.

```
package main

import (
	"fmt"
	"math"
)

func compute(fn func(float64, float64) float64) float64 {
	return fn(3, 4)
}

func main() {
	hypot := func(x, y float64) float64 {
		return math.Sqrt(x*x + y*y)
	}
	fmt.Println(hypot(5, 12))

	fmt.Println(compute(hypot))
	fmt.Println(compute(math.Pow))
}
```

# Function closures

Adder function returns a closure.
Each closure is bound to its own sum variable.

- Go functions may be closures.
- A closure is a function value that references variables from outside its body.
- The function may access and assign to the referenced variables; in this sense the function is "bound" to the variables.

```
package main
import "fmt"
func adder() func(int) int {
	sum := 0
	return func(x int) int {
		sum += x
		return sum
	}
}
func main() {
	pos, neg := adder(), adder()
	for i := 0; i < 10; i++ {
		fmt.Println(
			pos(i),
			neg(-2*i),
		)
	}
}
```

```
0 0
1 -2
3 -6
6 -12
10 -20
15 -30
21 -42
28 -56
36 -72
45 -90
.
```